



Karunadu Technologies Private Limited

#17, ATK complex, 4th Floor, Acharya college main road, beside Karur Vysya bank,
Guttebasaveshwaranagar Chikkabanvara, Bengaluru, Karnataka-560090

Internship Report

On

Python Programming and Applied ML

By

**Yashas TGS
(4JD22CS126)**

**BACHELOR OF ENGINEERING
COMPUTER SCIENCE AND ENGINEERING**

List of Contents

Topic	Pg. No
List of Figures	3
Chapter 1: Company Profile	4
1.1: Profile	4
1.1.1 : Vision	4
1.1.2 : Mission	4
1.1.3 : Objectives	5
1.2: Company Products and Services Offered	5
1.2.1 : Products	5
1.2.2 : Services	6
1.3: Contact Details	7
Chapter 2: Topics Learned During the Course	8
2.1: Python	8
2.1.1 : Features of Python	8
2.2: Python libraries for Data Analytics	9
2.2.1 : Numpy	9
2.2.2 : Pandas	10
2.2.3 : Matplotlib	11
2.3: Software Tool Used	12
2.4: Open CV	13
2.4.1 : Topics Covered	13
2.5: Machine Learning	18
2.5.1 : The Goals of Machine Learning	19
2.5.2 : Types of Machine Learning	19
2.5.3 : Applications of Python with Machine Learning	22

Chapter 3: Machine Learning Algorithm	23
3.1: Linear Regression	23
3.1.1 : Obtaining best fit line (Value of a and b)	24
3.1.2 : Advantages and Disadvantages	25
3.2: Multiple Linear Regression	27
3.2.1 : Advantages and Disadvantages	27
3.3: Logistic Regression	29
3.3.1 : Key Features	29
3.3.2 : Advantages and Disadvantages	30
3.4: KNN Regression	32
3.4.1 : Features Of KNN Algorithm	32
3.4.2 : Working	32
3.4.3 : Advantages and Disadvantages	34
3.5: SVM	36
3.5.1 : Features Of SVM	36
3.5.2 : Working	36
3.5.3 : Advantages and Disadvantages	37
3.6: Decision Tree	39
3.6.1 : Features Of Decision Tree	39
3.6.2 : Working	39
3.6.3 : Advantages nd Disadvantages	40
 Chapter 4: Industrial-Residential Air Quality Classification	 43
4.1 : Problem Statement	43
4.2 : Dataset	43
4.3 : Algorithm – Multiple Linear Regression	44
4.4 : Programming Steps	45
4.5 : Conclusion	47
 Chapter 5: Plant Communication	 48
5.1: Problem Statement	48
5.2: Dataset	49
5.3: Algorithm – KNN	50

5.4: Programming Steps	51
5.5: Conclusion	52
 Chapter 6: Smart Fertilizer Recommendation	 53
6.1: Problem Statement	53
6.2: Dataset	54
6.3: Algorithm – KNN	55
6.4: Programming Steps	55
6.5: Conclusion	56
 References	 57

List Of Figures

Sl. No	Fig. No	Name of the figure	Page No
1.	1.1	Company Logo	4
2.	2.1	Python Logo	8
3.	2.2	Pandas Data Frame components example	11
4.	2.3	Example of Gray and HSV Image	14
5.	2.4	Example of Resized Image	15
6.	2.5	Example of Added and Transparency Image	16
7.	2.6	Example of Rotated Image	17
8.	2.7	Example of Face and Eye Detection	18
9.	2.8	Machine Learning Infrastructure	18
10.	2.9	Flow Model of Algorithms of Supervised Learning	20
11.	2.10	Clustering and Regression	21
12.	2.11	Flow Model of Algorithms of Unsupervised Learning	21
13.	2.12	Flow Model of Algorithms of Reinforcement Learning	22
14.	3.1	Types of Machine Learning Algorithms	23
15.	3.2	Linear Regression	24
16.	3.3	Plot of Input Graph	24
17.	3.4	Example of Linear Regression	26
18.	3.5	Example of Multiple Linear Regression	28
19.	3.6	Linear Regression vs Logistic Regression	30
20.	3.7	Example of Logistic Regression	31
21.	3.8	Plots of ideal KNN Algorithm	33
22.	3.9	Example of KNN Algorithm	35
23.	3.10	Plot of ideal SVM Algorithm	37
24.	3.11	Example of SVM Algorithm	38
25.	3.12	Ideal diagram of a Decision Tree	40
26.	3.13	Example of Decision Tree	41
27.	4.1	Overview of Dataset	43
28.	4.2	Output of Yearly Amount Spent	45
29.	4.3	Implementing Yearly Amount Spent Prediction in PYQT5	46
30.	4.4	PYQT5 Output of Yearly Amount Spent Prediction	47
31.	5.1	Overview of Dataset	49
32.	5.2	Output of Car Name Prediction	51
33.	5.3	Implementing Car Name Prediction in PYQT5	52
34.	5.4	PYQT5 Output of Car Name Prediction	54

CHAPTER 1

Company Profile

It is pleasure in introducing “Karunadu Technologies Private Limited” as a leading IT software solutions and services industry focusing on quality standards and customer values. It is also a leading Skills and Talent Development company that is building a manpower pool for global industry requirements.

1.1 Profile



Fig 1.1 Company Logo

The company offers broad range of customized software applications powered by concrete technology and industry expertise. It also offers end to end embedded solutions and services. They deal with broad range of product development along with customized features ensuring at most customer satisfaction and also empower individual with knowledge, skills and competencies that assist them to escalate as integrated individuals with a sense of commitment and dedication.

1.1.1 Vision

To Empower Unskilled Individual with knowledge, skills and technical competencies in the field of Information Technology and Embedded engineering which assist them to escalate as integrated individuals contributing to company's and Nation's growth.

1.1.2 Mission

- Provide cost effective and reliable solutions to customers across various latest technologies.
- Offer scalable end-to-end application development and management solutions

- Provide cost effective highly scalable products for varied verticals.
- Focus on creating sustainable value growth through innovative solutions and unique partnerships.
- Create, design and deliver business solutions with high value and innovation by leveraging technology expertise and innovative business models to address long-term business objectives.
- Keep our products and services updated with the latest innovations in the respective requirement and technology.

1.1.3 Objectives

- To develop software and Embedded solutions and services focussing on quality standards and customer values.
- Offer end to end embedded solutions which ensure the best customer satisfaction.
- To build Skilled and Talented manpower pool for global industry requirements.
- To develop software and embedded products which are globally recognized.
- To become a global leader in Offering Scalable and cost-effective Software solutions and services across various domains like E-commerce, Banking, Finance, Healthcare and much more.
- To generate employment for skilled and highly talented youth of our Country INDIA.

1.2 Company Products and Services Offered

1.2.1 Products

- **KECMS – Karunadu Enterprise Content Management System**

Karunadu Enterprise Content Management System is a one stop solution for all our enterprise content management System relating to digital asset management, document imaging, workflow systems and records management systems. Increasing digitalization has led to an exponential growth in business content and managing this sea of unstructured data is tedious work. KECMS enables you to create, capture, manage, distribute, archive different forms of content and has much more features.

- **KEMS – Karunadu Education Management System**

Manage diversified data relating to education management on cloud. Educational data including students and staff is gathered over years which contain information from admission/appointment until leaving the Education. Statistical reports for the College/school can be generated along with admission Tracking and result analysis to keep track of progressive improvements of both student and staff.

- **KASS – Karunadu Advanced Security System**

A Complete one stop embedded solution for large apartments. Security system which monitors door breakage, window breakage, gas leakage, motion detection and various other features which can be operated and maintained by centralized monitored system. This Embedded solution enhances the security measures of apartment/building and enhances the security of individuals may be from unintended intervention or from unauthorized access.

1.2.2 Services

- **IT Solutions and Services**

Karunadu Technologies is a Bangalore based IT Training and Software Development center with an exclusive expertise in the area of IT Services and Solutions. Karunadu Technologies Pvt. Ltd. is also expertise in Web Designing and Consulting Services.

- **Embedded Design and Development**

Karunadu Technologies Pvt. Ltd. has expertise in Design and development of embedded products and offers solutions and services in field of Electronics.

- **Academic Projects**

Karunadu Technologies Pvt. Ltd. helps students in their academics by imparting industrial experience into projects to strive excellence of students. Karunadu Technologies Pvt. Ltd. encourages students to implement their own ideas to projects keeping in mind "A small seed sown upfront will be nourished to become a large tree one day", thereby focusing the future entrepreneurs. They have a wide range of IEEE projects for B.E, MTech, MCA, BCA, DIPLOMA students for all branches in each and every domain.

- **Implant Training**

Karunadu Technologies Pvt. Ltd. provides Implant training for students according to the interest of students keeping in mind the current technology and academic benefit one obtains after completing the training. Students will be nourished and will be trained throughout with practical experience. Students will be exposed to industrial standards which boost their carrier. Students will become Acquaint to various structural partitions such as labs, workshops, assembly units, stores, and administrative unit and machinery units. They help students to understand their functions, applications and maintenance. Students will be trained from initial stage that is from collection of Project Requirements, Project Planning, Designing, implementation, testing, deployment and maintenance there by helping to understand the business model of the industry. Entire project life cycle will be demonstrated with hands on experience. Students will also be trained about management skills and team building activities. They assure that by end of implant training students will Enhance communication skills and acquire technical skills, employability skills, start-up skills, and will be aware of risks in industry, management skills and many other skills which are helpful to professional engagement.

- **Software Courses**

Karunadu Technologies Pvt. Ltd. provides courses for students according to the interest of students keeping in mind the current technology and assist them for their further Employment. Company provides various courses such as C, C++, VB, DBMS, Dot Net, Core Java and J2EE along with live projects.

1.3 Contact Details



#17, ATK complex, 4th Floor, Acharya College Main Road, Beside KarurVysya Bank, Guttebasaveshwaranagar, Chikkabanvara, Bengaluru, Karnataka- 560090



support@karunadutechnologies.com

CHAPTER 2

Topics Learnt During the Course

The objective of the internship is to apply theoretical knowledge of “Machine Learning using Python” to solve real time complex problems, in order to achieve these following basic concepts were learnt:

- Python
- Machine Learning

2.1 Python

Python is a multiparadigm, general-purpose, interpreted, high-level programming language. Python allows programmers to use different programming styles to create simple or complex programs, get quicker results and write code almost as if speaking in a human language.



Fig 2.1 Python Logo

The topics learnt on Python are as follows:

- Installation of Python.
- Use of variables to store, retrieve and calculate information.
- Utilization of core programming operations such as functions and loops.
- Operation on strings, python supported libraries for Machine Learning.

2.1.1 Features of Python

- Extensive support libraries (NumPy for numerical calculations, Pandas for data analytics).
- Open source and community development.

- Dynamically typed language (No need to mention data type based on value assigned, it takes data type).
- Object-oriented language, Portable and Interactive across Operating systems.

2.2 Python libraries for Data Analytics

Machine Learning, as the name suggests, is the science of programming a computer by which they are able to learn from different kinds of data. A more general definition given by Arthur Samuel is – “Machine Learning is the field of study that gives computers the ability to learn without being explicitly programmed.” They are typically used to solve various types of life problems.

In the older days, people used to perform Machine Learning tasks by manually coding all the algorithms and mathematical and statistical formula. This made the process time consuming, tedious and inefficient. But in the modern days, it is become very much easy and efficient compared to the olden days by various python libraries, frameworks, and modules. Python libraries that used in Machine Learning are:

- NumPy
- Pandas
- Matplotlib

2.2.1 Numpy

Numpy is basic package for scientific computing. It is the python language implementation which includes powerful N-dimensional array structure, sophisticated functions, Tools that can be integrated into C/C++ and Fortran code, Linear algebra, Fourier transform and Random number features. Besides its obvious scientific uses, numpy can also be used as an efficient multidimensional container of generic data.

The main aspect Numpy is the Numpy array, on which you can do various operations. The key is that a Numpy array isn't just a regular array you'd see in a language like Java or C++, but instead it is like a mathematical object as a vector or a matrix. That means you can do vector and matrix operations like addition, subtraction, and multiplication. The most important aspect of NumPy arrays is that they are optimized for speed.

NumPy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays. It is the fundamental package for scientific computing with Python. Besides its obvious scientific uses, NumPy can also be used as an efficient multi-dimensional container of generic data.

- **Basic Array Operations**

NumPy, arrays allow a wide range of operations which can be performed on a particular array or a combination of Arrays. These operations include some basic Mathematical operation as well as Unary and Binary operations.

```
# Python program to demonstrate
# basic operations on single array
import NumPy as np
a = np.array([[1, 2], [3, 4]])# Defining Array 1
b = np.array([[4, 3], [2, 1]]) # Defining Array 2
print ("Adding 1 to every element:", a + 1)# Adding 1 to every element
print ("\n Subtracting 2 from each element:", b - 2)# Subtracting 2 from each element
# sum of array elements
print ("\n Sum of all array "elements: ", a.sum()) # Performing Unary operations
print ("\n Array sum:\n", a + b)# Performing Binary operations
```

2.2.2Pandas

Pandas Data Frame is two-dimensional size-mutable, potentially heterogeneous tabular data structure with labelled axes (rows and columns). A Data frame is a two-dimensional data structure, i.e., data is aligned in a tabular fashion in rows and columns. Pandas Data Frame consists of three principal components, the data, rows, and columns as shown in the fig 2.2.

The basic operations which can be performed on Pandas Data Frame are:

- Creating a Data Frame
- Dealing with Rows and Columns
- Indexing and Selecting Data
- Working with Missing Data

	Name	Team	Number	Position	Age
0	Avery Bradley	Boston Celtics	0.0	PG	25.0
1	John Holland	Boston Celtics	30.0	SG	27.0
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN
6	Evan Turner	Boston Celtics	11.0	SG	27.0

Fig 2.2 Pandas Data Frame components example

#Python program using Pandas for arranging a given set of data into a table

```
import pandas as pd
```

```
data = {"country": ["Brazil", "Russia", "India", "China", "South Africa"],
        "capital": ["Brasilia", "Moscow", "New Delhi", "Beijing", "Pretoria"],
        "area": [8.516, 17.10, 3.286, 9.597, 1.221],
        "population": [200.4, 143.5, 1252, 1357, 52.98] }
```

```
data table = pd. DataFrame(data)
```

```
print(data table)
```

2.2.3 Matplotlib

Matplotlib is a very popular Python library for data visualization. Like Pandas, it is not directly related to Machine Learning. It particularly comes in handy when a programmer wants to visualize the patterns in the data. It is a 2D plotting library used for creating 2D graphs and plots. A module named pyplot makes it easy for programmers for plotting as it provides features to control line styles, font properties, formatting axes, etc. It provides various kinds of graphs and plots for data visualization, viz., histogram, error charts, bar charts, etc,

#Python program using Matplotlib for forming a linear plot

```
import matplotlib. Pyplot as plt# importing the necessary packages and modules
```

```
import NumPy as np
```

```
x = np. linspace (0, 10, 100)# Prepare the data
```

```
plt. Plot (x, x, label ='linear' )# Plot the data
```

```
plt. Legend() # Add a legend
```

```
plt. Show()# Show the plot
```

2.3 Software tool used

Anaconda is a free and open-source distribution of the Python programming language for scientific computing (data science, machine learning applications, large-scale data processing, predictive analytics, etc.), that aims to simplify package management and deployment. Package versions are managed by the package management system conda. Anaconda distribution comes with more than 1,500 packages as well as the conda package and virtual environment manager. It also includes a GUI, Anaconda Navigator, as a graphical alternative to the command line interface (CLI). Anaconda Navigator can search for packages on Anaconda Cloud or in a local Anaconda Repository, install them in an environment, run the packages and update them. There are many applications available by default in navigator; among them is the Spyder.

Spyder is an open source cross-platform integrated development environment (IDE) for scientific programming in the Python language. Spyder integrates with a number of prominent packages. Some of the features of Spyder are:

- An editor with syntax highlighting, introspection, code completion.
- Support for multiple I Python consoles.
- The ability to explore and edit variables from a GUI.
- A Help pane able to retrieve and render rich text documentation on functions, classes and methods automatically or on-demand.
- A debugger linked to IP dB, for step-by-step execution.
- A run-time Profiler, to benchmark code.
- Project support, allowing work on multiple development efforts simultaneously.
- A built-in file explorer, for interacting with the filesystem and managing projects.
- A "Find in Files" feature, allowing full regular expression search over a specified scope.
- An online help browser, allowing users to search and view Python and package documentation inside the IDE.
- A history log, recording every user command entered in each console.
- An internal console, allowing for introspection and control over Spyder's own operation.

2.4 Open CV

OpenCV is a huge open-source library for computer vision, machine learning, and image processing. OpenCV supports a wide variety of programming languages like Python, C++, Java, etc. It can process images and videos to identify objects, faces, or even the handwriting of a human. When it is integrated with various libraries, such as Numpy which is a highly optimized library for numerical operations, then the number of weapons increases in your Arsenal i.e., whatever operations one can do in Numpy can be combined with OpenCV.

Definition

OpenCV (Open-Source Computer Vision Library) is an open-source computer vision and machine learning software library. OpenCV was built to provide a common infrastructure for computer vision applications and to accelerate the use of machine perception in the commercial products. Being a BSD-licensed product, OpenCV makes it easy for businesses to utilize and modify the code.

2.4.1 Topics Covered

- **Converting Image to Grey scale and HSV scale**

Here we convert the given image into a grey scale image i.e., Black and white image. The function used here is `cvtColor` and the code used for grey scale is `'COLOR_BGR2GRAY'`. Similarly, we do for HSV scale and the function used is `'COLOR_BGR2HSV'`. Following is an example for the above scales.

CODE:

```
import cv2
path=" C:\\Users\\Admin\\Desktop\\ajay\\Images\\cats.jpg"
img = cv2.imread(path)
cv2.imshow("Original Image", img)
img1= cv2.resize(img,(512,512))
cv2.imshow("Resized Image",img1)
grayimg = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
grayimg= cv2.resize(grayimg,(512,512))
```

```
cv2.imshow("Gray Image", grayimg)
having = cv2.cvtColor(img,cv2.COLOR_BGR2HSV)
having= cv2.resize(having,(512,512))
cv2.imshow(" HSV Image", having)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

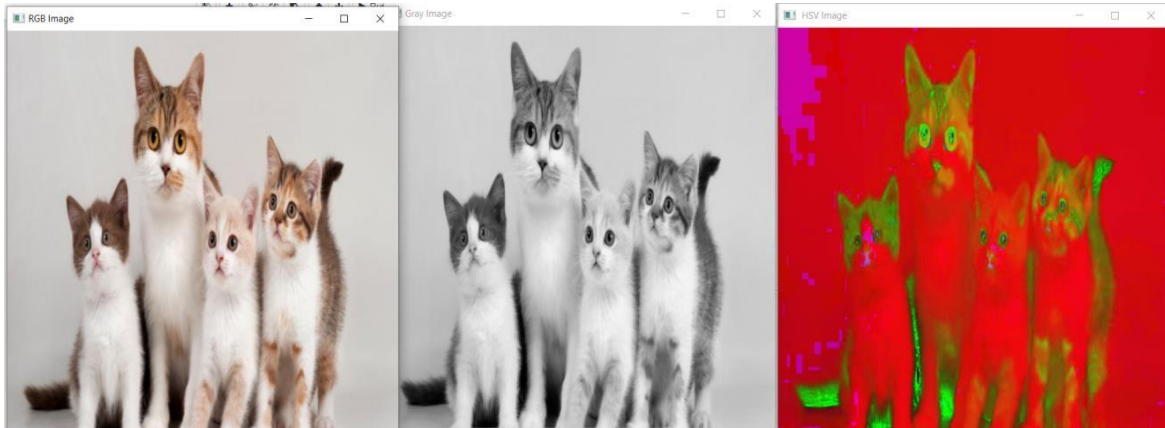


Fig 2.3 Example of Grey and HSV Image

- **Resizing Image**

Here we can resize the original image into whichever size required by us. The function used for resizing is 'resize'. For further applications we require resizing of images in order to match them for further uses. This is particularly useful for cascading more than two images.

CODE:

```
import cv2
path = "C:\\Users\\Admin\\Desktop\\ajay\\Images\\Image.png"
img = cv2.imread(path)
cv2.imshow("Original Image", img)
img1= cv2.resize(img,(512,512))
cv2.imshow("Resized Image",img1)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Fig 2.4 Example of Resized Image

- **Adding Images and Transparency**

Here in order cascade or adding the images we require the images to be of the same size. We use resize function to resize the image. Here adding the images means in single image we paste them with required transparency. We can adjust the transparency according to our demand.

CODE:

```
import cv2
path1="C:\\Users\\Admin\\Desktop\\ajay\\Images\\Image.png"
img1 = cv2.imread(path1)
path2="C:\\Users\\Admin\\Desktop\\ajay\\Images\\cats.jpg"
img2 = cv2.imread(path2)
img1 = cv2.resize(img1,(512,512))
img2 = cv2.resize(img2,(512,512))
addimages = cv2.add(img1,img2)
addtransperency = cv2.addWeighted(img1,0.8,img2,0.2,0)
cv2.imshow("First Image",img1)
cv2.imshow("Second Image",img2)
cv2.imshow("Added Image", addimages)
cv2.imshow("Added Image with transparency", addtransperency)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

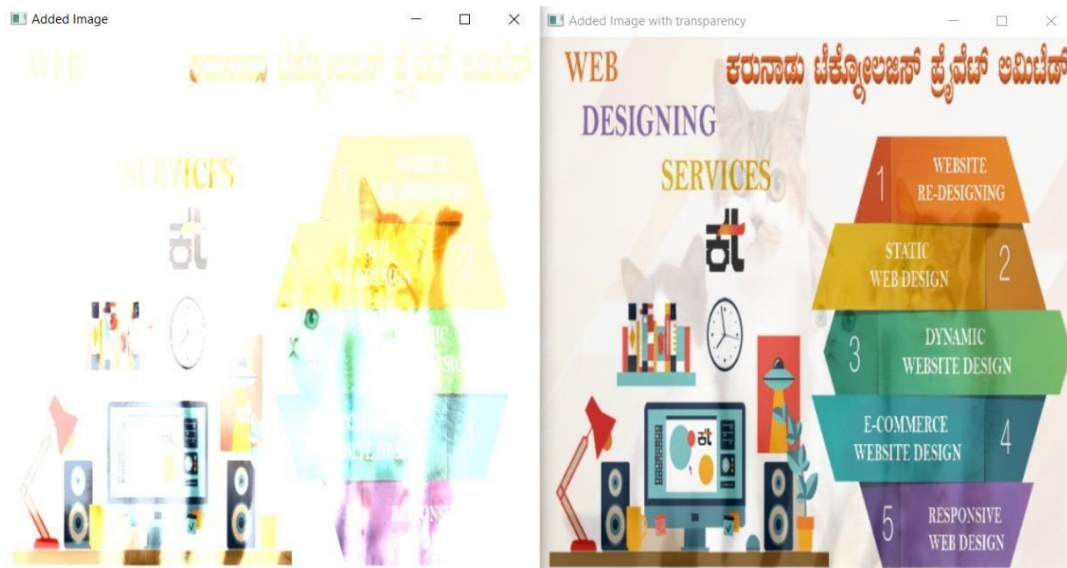


Fig 2.5 Example of Added and Transparency Image

- **Rotating Images**

Here we rotate the images in clockwise or anticlockwise direction in the required degree. The function we use 'rotate' in order to rotate the images.

CODE:

```
import cv2

path="C:\\Users\\Admin\\Desktop\\ajay\\Images\\cats.jpg"
img = cv2.imread(path)
imgresize = cv2.resize(img, (300,300))
img90 = cv2.rotate(imgresize, cv2.ROTATE_90_CLOCKWISE)
img180 = cv2.rotate(imgresize, cv2.ROTATE_180)
img270 = cv2.rotate(imgresize, cv2.ROTATE_90_COUNTERCLOCKWISE)
cv2.imshow("Original Image", img)
cv2.imshow("90degree Image", img90)
cv2.imshow("180degree Image", img180)
cv2.imshow("270degree Image", img270)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

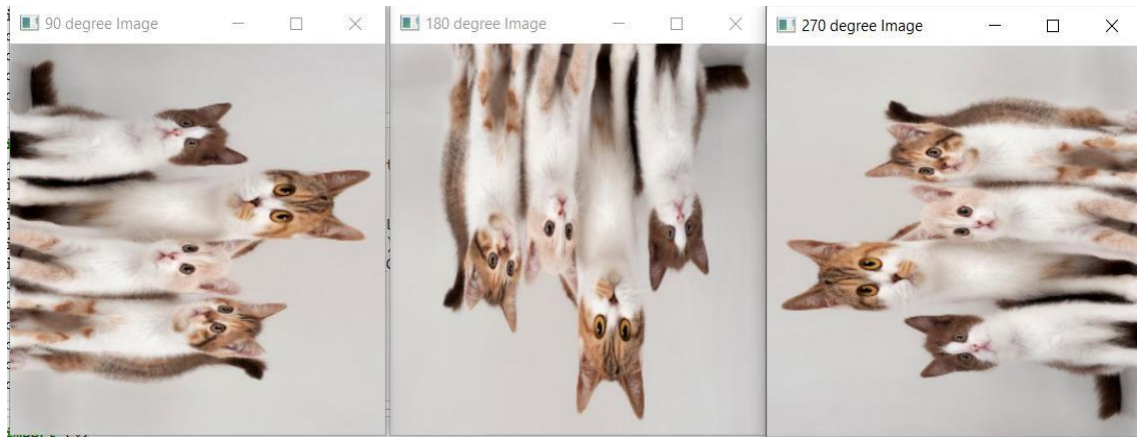


Fig 2.6 Example of Rotated Image

- **Face and Eye Detection**

Here we detect the face and eye of an individual while he is alone or in a group. It is possible to detect more than one face and eye at a time. Here we use haarcascades which is a pre-defined function in order to detect the faces and eyes. It is also possible to detect the faces and eyes in image or video. Similarly, we can do live screening and also, we need to define different function for various tasks.

CODE:

```
import cv2
path="C:\\Users\\Admin\\Desktop\\ajay\\Images\\rotated_face.jpeg"
img = cv2.imread(path)
face_cascade=cv2.CascadeClassifier(cv2.data.haarcascades+'haarcascade_frontalface_default.xml')
faces = face_cascade. detectMultiScale(img,1.1,4)
for (x, y, w, h) in faces:
cv2.rectangle(img, (x, y),(x+w,y+h),(255,255,0),8)
cv2.imshow("Face detected", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

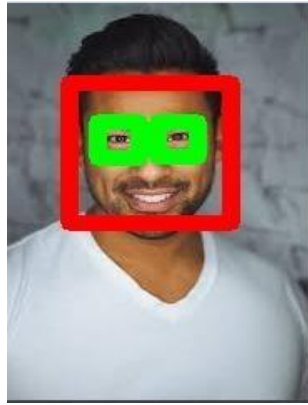


Fig 2.7 Example of Face and Eye Detection.

2.5 Machine Learning

Machine learning (ML) is a type of artificial intelligence (AI) that allows software applications to become more accurate at predicting outcomes without being explicitly programmed to do so. Machine learning algorithms use historical data as input to predict new output values.

Definition: Ability of a machine to improve its own performance through the use of software that employs artificial intelligence techniques to mimic the ways by which humans seem to learn, such as repetition and experience.

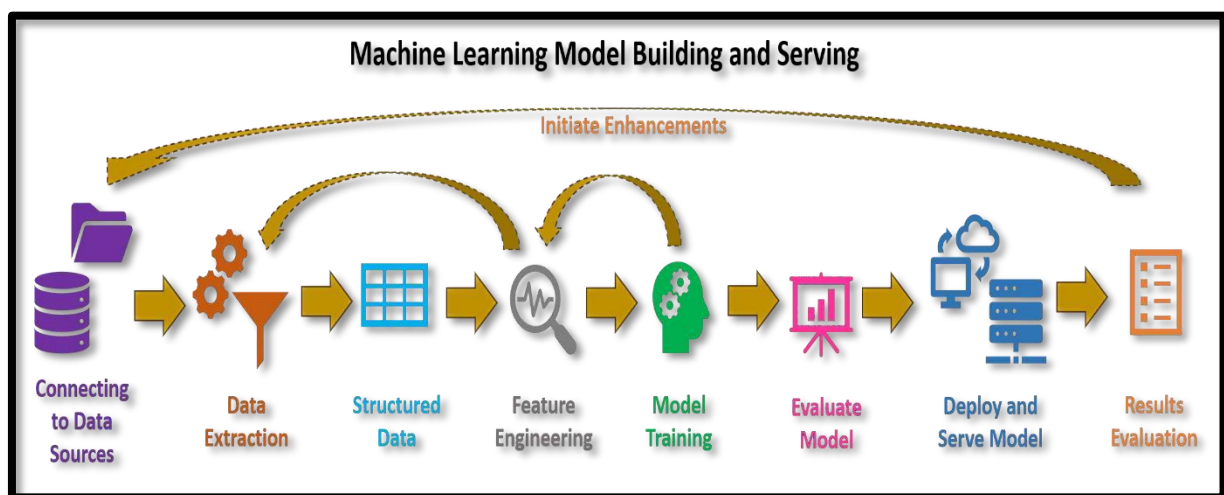


Fig 2.8 Machine Learning Infrastructure

Machine Learning (ML) is a sub-field of Artificial Intelligence (AI) which concerns with developing computational theories of learning and building learning machines. This sequence of learning and building is as shown in fig 2.9. The goal of machine learning, closely coupled with the goal of AI, is to achieve a thorough understanding about the nature

of learning process (both human learning and other forms of learning), about the computational aspects of learning behaviours, and to implant the learning capability in computer systems.

2.5.1 The Goals of Machine Learning

The goal of ML, in simple words, is to understand the nature of human and other forms of learning, and to build learning capability in computers. To be more specific, there are three aspects of the goals of ML.

- To make the computers smarter, more intelligent. The more direct objective in this aspect is to develop systems (programs) for specific practical learning tasks in application domains.
- To develop computational models of human learning process and perform computer simulations. The study in this aspect is also called cognitive modelling.
- To explore new learning methods and develop general learning algorithms independent of applications.

2.5.2 Types of Machine Learning

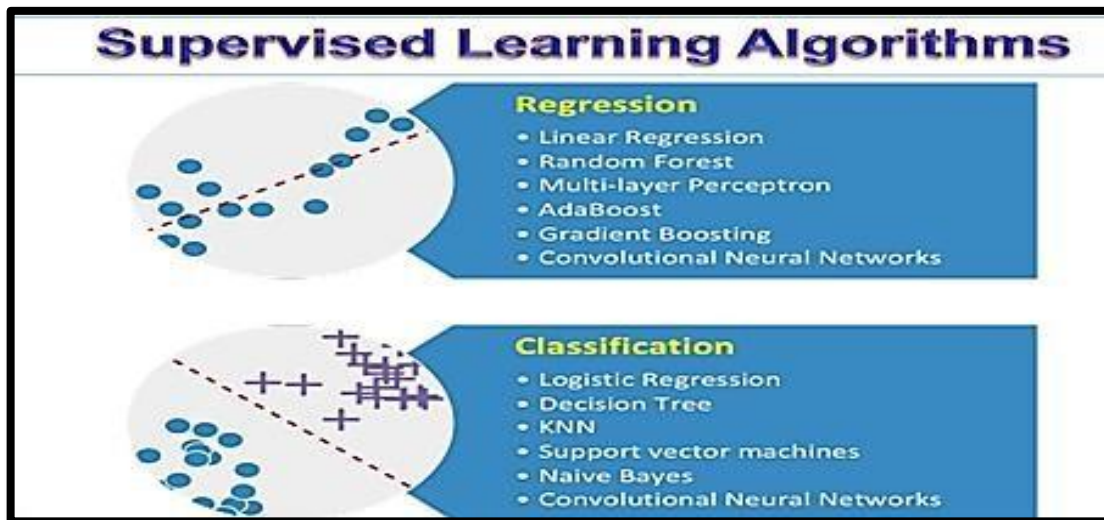
➤ Supervised learning

Supervised learning is a type of machine learning method in which we provide sample labelled data to the machine learning system in order to train it, and on that basis, it predicts the output. The system creates a model using labelled data to understand the datasets and learn about each data, once the training and processing are done then we test the model by providing a sample data to check whether it is predicting the exact output or not.

The goal of supervised learning is to map input data with the output data. The supervised learning is based on supervision, and it is the same as when a student learns things in the supervision of the teacher. The example of supervised learning is **spam filtering**.

Supervised learning can be grouped further in two categories of algorithms:

- **Classification**: A classification problem is when the output variable is a category, such as “Red” or “blue” or “disease” and “no disease”.
- **Regression**: A regression problem is when the output variable is a real value, such as “dollars” or “weight”.



2.9 Flow Model of Algorithms of Supervised Learning.

➤ Unsupervised Learning

Unsupervised learning is the training of machine using information that is neither classified nor labelled and allowing the algorithm to act on that information without guidance. Here the task of machine is to group unsorted information according to similarities, patterns and differences without any prior training of data. Unlike supervised learning, no teacher is provided that means no training will be given to the machine. Therefore, machine is restricted to find the hidden structure in unlabelled data by our-self.

For instance, suppose it is given an image having both dogs and cats which have not seen ever. Thus, machine has no any idea about the features of dogs and cat so we can't categorize it in dogs and cats. But it can categorize them according to their similarities, patterns and differences i.e., one can easily categorize the above picture into two parts. First may contain all pics having dogs in it and second part may contain all pics having cats in it.

Unsupervised learning classified into two categories of algorithms:

- **Clustering**: A clustering problem is where one wants to discover the inherent groupings in the data as shown in fig 2.11, such as grouping customers by purchasing behaviour.
- **Association**: An association rule learning problem is where one wants to discover rules that describe large portions of the data, such as people that buy X also tend to buy Y.

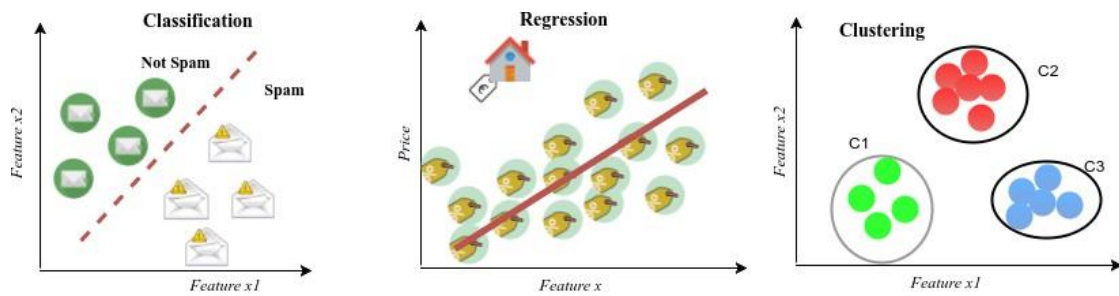
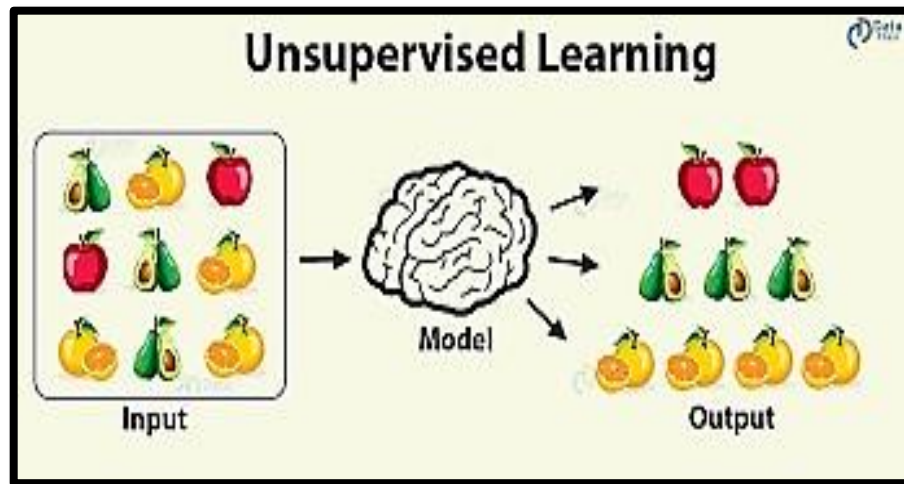


Fig 2.10 Clustering and Regression



2.11 Flow Model of Algorithms of Unsupervised Learning.

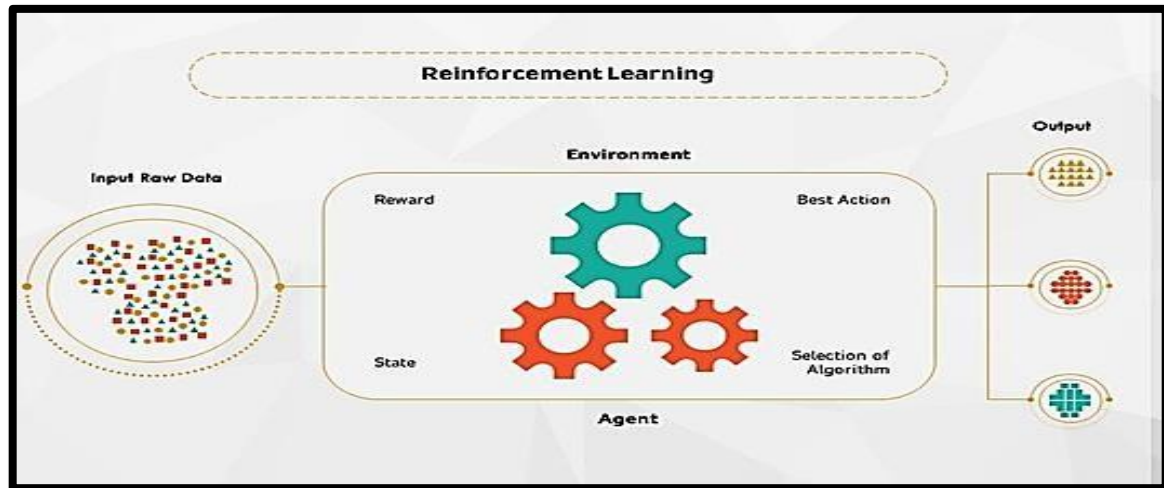
➤ Reinforcement learning

Reinforcement learning is an area of Machine Learning. It is about taking suitable action to maximize reward in a particular situation. It is employed by various software and machines to find the best possible behaviour or path it should take in a specific situation. Reinforcement learning differs from the supervised learning in a way that in supervised learning the training data has the answer key with it so the model is trained with the correct answer itself whereas in reinforcement learning, there is no answer but the reinforcement agent decides what to do to perform the given task. In the absence of training dataset, it is bound to learn from its experience.

There are two types of Reinforcement learning:

- **Positive:** Positive Reinforcement is defined as when an event, occurs due to a particular behaviour, increases the strength and the frequency of the behaviour. In other words, it has a positive effect on the behaviour. It maximizes performance and sustains change for a long period of time. Too much Reinforcement can lead to overload of states which can diminish the results.

- **Negative:** Negative Reinforcement is defined as strengthening of behaviour because a negative condition is stopped or avoided. It increases behaviour and provide defiance to minimum standard of performance. It only provides enough to meet up the minimum behaviour.



2.13 Flow Model of Algorithms of Reinforcement Learning.

2.5.3 Applications of Machine Learning

Machine learning has been recognized as central to the success of Artificial Intelligence, and it has applications in various areas of science, engineering and society. Some of them are:

- Product recommendations (e.g., Amazon etc.)
- Refining the search engine results(e.g., Google)
- Fighting the web spam(e.g., Gmail)
- Video surveillance(e.g., crime alerts)
- Face recognition and many more.

CHAPTER 3

Machine Learning Algorithms

Machine learning is an application of artificial intelligence (AI) that provides systems the ability to automatically learn and improve from experience without being explicitly programmed. Machine learning focuses on the development of computer programs that can access data and use it learn for themselves.

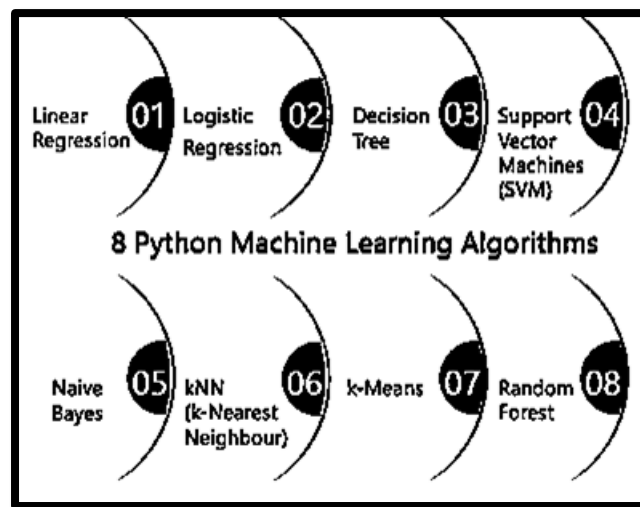


Fig 3.1 Types of Machine Learning Algorithms

3.1 Linear Regression

It is one of the most widely known modeling technique. Linear regression is usually among the first few topics which people pick while learning predictive modelling. In this technique, the dependent variable is continuous, independent variable(s) can be continuous or discrete, and nature of regression line is linear. Linear Regression establishes a relationship between dependent variable (Y) and one or more independent variables(X) using a best fit straight line (also known as regression line) as shown in fig 3.2. It is represented by an equation $Y = a + b \cdot X + e$, where a is intercept, b is slope of the line and e is error term. This equation can be used to predict the value of target variable based on given predictor variable.

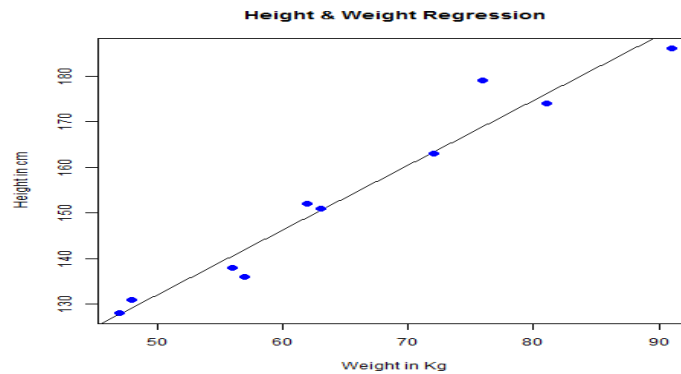
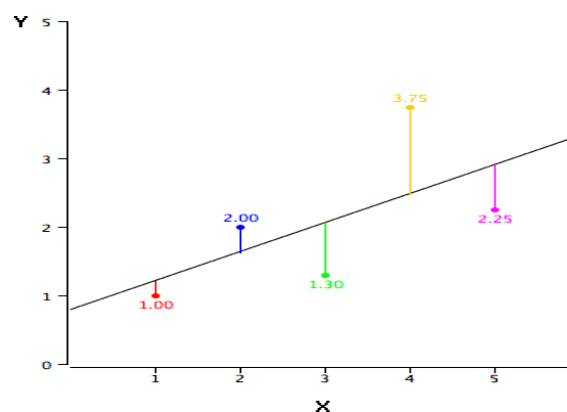


Fig 3.2 Linear Regression

3.1.1 Obtaining best fit line (Value of a and b)

This task can be easily accomplished by Least Square Method. It is the most common method used for fitting a regression line. It calculates the best-fit line for the observed data by minimizing the sum of the squares of the vertical deviations from each data point to the line. Because the deviations are first squared, when added, there is no cancelling out between positive and negative values.



$$\min_w ||Xw - y||_2^2$$

Fig 3.3 Plot of Input Graph

There must be linear relationship between independent and dependent variables. Linear Regression is very sensitive to Outliers. It can terribly affect the regression line and eventually the forecasted values. Simple linear regression is used for finding the relationship between the dependent variable Y and the independent or predictor variable X. Both of these variables are continuous in nature. While performing simple linear regression, we assume that the values of

predictor variable X are controlled. Furthermore, they are not subject to the measurement error from which the corresponding value of Y is observed.

The equation of a simple linear regression model to calculate the value of the dependent variable, Y based on the predictor X is as follows:

$$y_i = b_0 + b_1 x + e_i$$

Where the value of y_i is calculated with the input variable x_i for every i th data point; The coefficients of regressions are denoted by b_0 and b_1 ; the i th value of x has e_i as its error in the measurement.

Regression analysis is implemented to do the following:

- With it, we can establish a linear relationship between the independent and the dependent variables.
- The input variables $x_1, x_2, \dots, x_n$ is responsible for predicting the value of y .
- In order to explain the dependent variable precisely, we need to identify the independent variables carefully. This will allow us to establish a more accurate causal relationship between these two variables.

3.1.2 Advantages and Disadvantages

Advantages

Linear regression is an extremely simple method. It is very easy and intuitive to use and understand. A person with only the knowledge of high school mathematics can understand and use it. In addition, it works in most of the cases. Even when it doesn't fit the data exactly, we can use it to find the nature of the relationship between the two variables.

Disadvantages

- By its definition, linear regression only models relationships between dependent and independent variables that are linear. It assumes there is a straight-line relationship between them which is incorrect sometimes. Linear regression is very sensitive to the anomalies in the data (or outliers).
- Take for example most of your data lies in the range 0-10. If due to any reason only one of the data items comes out of the range, say for example 15, this significantly influences the regression coefficients.

- Another disadvantage is that if we have a number of parameters than the number of samples available then the model starts to model the noise rather than the relationship between the variables.

```
def AREA(self):  
    import pandas as pd  
    import matplotlib.pyplot as plt  
    from sklearn import linear_model  
  
    path="C:\\Users\\Admin\\Desktop\\ajay\\Data\\homeprices.csv"  
  
    data=pd.read_csv(path)  
  
    inputdata=data.drop('price','columns')  
    output=data.drop('area','columns')  
  
    model=linear_model.LinearRegression()  
    model.fit(inputdata,output)  
  
    a=float(self.txtarea.text())  
  
    self.lblmessage.setText(str(model.predict([[a]])))
```

HOUSE PRICE PREDICTION

AREA

PREDICT

[[533664.38356164]]

Fig 3.4 Example of Linear Regression

3.2 Multiple Linear Regression

In many cases, there may be possibilities of dealing with more than one predictor variable for finding out the value of the response variable. Therefore, the simple linear models cannot be utilized as there is a need for undertaking multiple linear regression for analyzing the predictor variables. The difference between simple linear regression and multiple linear regression is that, multiple linear regression has more than 1 independent variables, whereas simple linear regression has only 1 independent variable. Using the two explanatory variables, we can delineate the equation of multiple linear regression as follows:

$$y_i = b_0 + b_1x_1 + b_2x_2 + e_i$$

The two explanatory variables x_1 and x_2 , determine y_i , for the i th data point. Furthermore, the predictor variables are also determined by the three parameters b_0 , b_1 , and b_2 of the model, and by the residual e_i of the point i from the fitted surface. General Multiple regression models can be represented as:

$$y_i = \sum b_i x_i + e_i$$

Multiple regression suffers from multicollinearity, autocorrelation, heteroskedasticity. Multicollinearity can increase the variance of the coefficient estimates and make the estimates very sensitive to minor changes in the model. The result is that the coefficient estimates are unstable. In case of multiple independent variables, we can go with forward selection, backward elimination and step wise approach for selection of most significant independent variables.

3.2.1 Advantages and disadvantages

Advantages

- The ability to determine the relative influence of one or more predictor variables to the criterion value.
- The ability to identify outliers, or anomalies.

Disadvantage

- Any disadvantage of using a multiple regression model usually comes down to the data being used. Two examples of this are using incomplete data and falsely concluding that a correlation is causation.

```
def profit(self):
    import pandas as pd
    import matplotlib.pyplot as plt
    import sklearn
    from sklearn import linear_model
    path = "C:\\Users\\Admin\\Desktop\\ajay\\Data\\50_Startups.csv"
    data = pd.read_csv(path)
    print(data)
    inputs=data.drop('Profit','columns')
    print(inputs)
    outputs=data.drop(['R&D Spend','Administration','Marketing Spend'],'columns')
    print(outputs)
    model = linear_model.LinearRegression()
    model.fit(inputs,outputs)

    RDSpend = self.txttrdspond.text()
    Administration = self.txtadministration.text()
    MarketingSpend = self.txtmarketingspend.text()

    ans = model.predict([[int(RDSpend),int(Administration),int(MarketingSpend)]])

    self.lblmessage.setText(str(ans))
```

predicting the profit using multiple linear regression

R&D Spend	2560
Administration	25
Marketing Spend	2569

[[52254.10201691]]

Fig 3.5 Example of Multiple Linear Regression

3.3 Logistic Regression

Logistic regression is a statistical model that in its basic form uses a logistic function to model a binary dependent variable, although many more complex extensions exist. In regression analysis, logistic regression is estimating the parameters of a logistic model (a form of binary regression). Mathematically, a binary logistic model has a dependent variable with two possible values, such as pass/fail which is represented by an indicator variable, where the two values are labelled "0" and "1". In the logistic model, the log-odds (the logarithm of the odds) for the value labelled "1" is a linear combination of one or more independent variables ("predictors"); the independent variables can each be a binary variable (two classes, coded by an indicator variable) or a continuous variable (any real value). The corresponding probability of the value labelled "1" can vary between 0 (certainly the value "0") and 1 (certainly the value "1"), hence the labelling, the function that converts log-odds to probability is the logistic function, hence the name. The unit of measurement for the log-odds scale is called a logit, from logistic unit, hence the alternative names. Analogous models with a different sigmoid function instead of the logistic function can also be used, such as the probit model; the defining characteristic of the logistic model is that increasing one of the independent variables multiplicatively scales the odds of the given outcome at a constant rate, with each independent variable having its own parameter; for a binary dependent variable this generalizes the odds ratio.

3.3.1 Key Features

- Logistic regression predicts whether something is True(1) or False(0) instead, predicting something that is continuous like size.
- It has an S-shaped line.
- We can take our Linear Regression Model and convert it into Logistic Regression model with the help of Sigmoid Function.
- Logistic Regression's ability to provide probabilities and classify new samples using continuous and discrete measurements makes it a popular machine learning method.

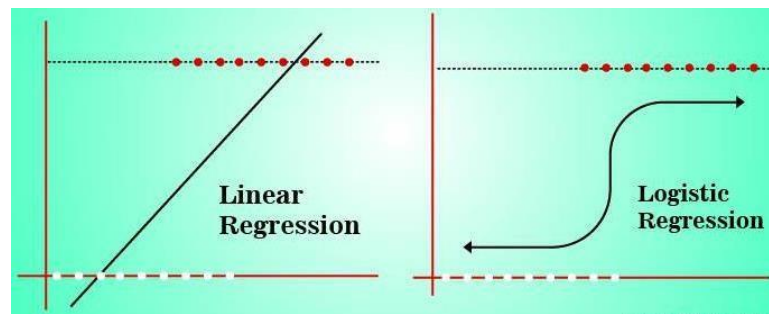


Fig 3.6 Linear Regression v/s Logistic Regression

This is where logistic regression comes into play. In logistic regression, you get a probability score that reflects the probability of the occurrence of the event. An event in this case is each row of the training dataset. It could be something like classifying if a given email is spam, or mass of cell is malignant or a user will buy a product and so on.

3.3.2 Advantages and Disadvantages

Advantages

- It doesn't require high computational power.
- Is easily interpretable.
- Is used widely by the data analyst and data scientists.
- Is very easy to implement.
- It doesn't require scaling of features.
- It provides a probability score for observations.

Disadvantages

- While working with Logistic regression you are not able to handle a large number of categorical features/variables.
- It is vulnerable to over fitting.
- It can't solve the non-linear problem with the logistic regression model that is why it requires a transformation of non-linear features.
- Logistic regression will not perform well with independent(X) variables that are not correlated to the target(Y) variable.

```
def buy(self):
    import pandas as pd
    path = "C:\\Users\\Admin\\Desktop\\ajay\\LogisticRegression\\Magzine\\Kid.csv"
    data = pd.read_csv(path)
    print(data)

    inputs = data.drop(['Buy', 'Obs No'], 'columns')
    output = data['Buy']
    print(inputs)
    print(output)

    import sklearn
    from sklearn.model_selection import train_test_split
    x_train, x_test, y_train, y_test = train_test_split(inputs, output, test_size=0.2)
    from sklearn.linear_model import LogisticRegression
    model = LogisticRegression()
    model.fit(x_train, y_train)
    y_pred = model.predict(x_test)
    print(y_pred)
    from sklearn.metrics import confusion_matrix
    cm = confusion_matrix(y_test, y_pred)
    print(cm)
    Income = self.txtincome.text()
    IsFemale = self.txtisfemale.text()
    IsMarried = self.txtismarried.text()
    HasCollege = self.txthascollege.text()
    IsProfessional = self.txtisprofessional.text()
    IsRetired = self.txtisretired.text()
    Unemployed = self.txtunemployed.text()
    ResidenceLength = self.txtresidencelength.text()
    DualIncome = self.txtdualincome.text()
    Minors = self.txtminors.text()
    Own = self.txtown.text()
    House = self.txthouse.text()
    White = self.txtwhite.text()
    English = self.txtenglish.text()
    PrevChildMag = self.txtprevchildmag.text()
    PrevParentMag = self.prevprenmag.text()
    ans = model.predict([int(Income), int(IsFemale), int(IsMarried), int(HasCollege), int(IsProfessional),
                          int(IsRetired), int(Unemployed), int(ResidenceLength), int(DualIncome),
                          int(Minors), int(Own), int(House), int(White), int(English), int(PrevChildMag), int(PrevParentMag))])
    if ans==0:
        self.lblmessage.setText(str("YES"))
    elif ans == 1:
        self.lblmessage.setText(str("NO"))
```

MAGZINE BUY OR NOT			
INCOME	7500	DUAL INCOME	1
IS FEMALE	1	MINORS	0
IS MARRIED	1	OWN	1
HAS COLLEGE	1	HOUSE	1
IS PROFESSIONAL	1	WHITE	1
IS RETIRED	0	ENGLISH	1
UNEMPLOYED	0	PREV CHILD MAG	1
RESIDENCE LENGTH	15	PREV PARENT MAG	0
PREDICT			
NO			

MAGZINE BUY OR NOT			
INCOME	2400	DUAL INCOME	0
IS FEMALE	1	MINORS	0
IS MARRIED	0	OWN	0
HAS COLLEGE	1	HOUSE	1
IS PROFESSIONAL	1	WHITE	0
IS RETIRED	0	ENGLISH	0
UNEMPLOYED	0	PREV CHILD MAG	0
RESIDENCE LENGTH	26	PREV PARENT MAG	0
PREDICT			
YES			

Fig 3.7 Example of Logistic Regression

3.4 KNN

K nearest neighbors or KNN Algorithm is a simple algorithm which uses the entire dataset in its training phase. Whenever a prediction is required for an unseen data instance, it searches through the entire training dataset for k-most similar instances and the data with the most similar instance is finally returned as the prediction. KNN is often used in search applications where you are looking for similar items, like find items similar to this one.

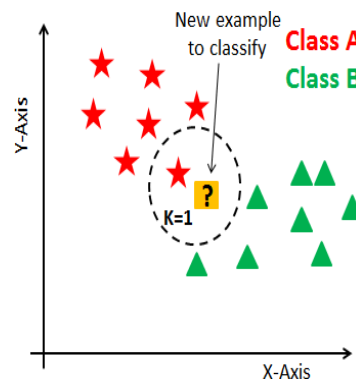
3.4.1 Features of KNN Algorithm

- KNN is a Supervised Learning algorithm that uses labelled input data set to predict the output of the data points.
- It is one of the simplest Machine learning algorithms and it can be easily implemented for a varied set of problems.
- It is mainly based on feature similarity. KNN checks how similar a data point is to its neighbor and classifies the data point into the class it is most similar to.
- Unlike most algorithms, KNN is a non-parametric model which means that it does not make any assumptions about the data set. This makes the algorithm more effective since it can handle realistic data.
- KNN is a lazy algorithm; this means that it memorizes the training data set instead of learning a discriminative function from the training data.
- KNN can be used for solving both classification and regression problems.

3.4.2 Working

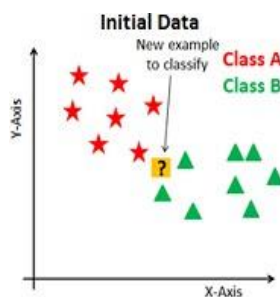
In KNN, K is the number of nearest neighbors. The number of neighbors is the core deciding factor. K is generally an odd number if the number of classes is 2. When K=1, then the algorithm is known as the nearest neighbor algorithm. This is the simplest case. However, the number of neighbours (K) is a hyper parameter that needs to be chosen at the time of model building. Research has shown that no optimal number of neighbors suits all kind of data sets. Each dataset has its own requirements. Generally, Data scientists choose as an odd number if the number of classes is even. We can also check by generating the model on different values of k and check their performance.

Suppose '?' is the point, for which label needs to predict. First, you find the k closest point to P1 and then classify points by majority vote of its k neighbors. Each object votes for their class and the class with the most votes are taken as the prediction. For finding closest similar points, you find the distance between points using distance measures such as Euclidean distance, Hamming distance, Manhattan distance and Minkowski distance. Then we find the one closest point to '?' and then the label of the nearest point is assigned to '?'.

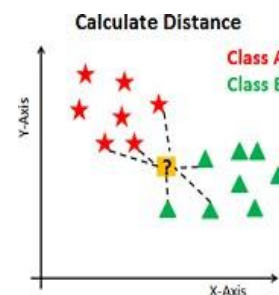


KNN has the following basic steps:

1. Calculate distance



2. Find closest neighbors



3. Vote for labels

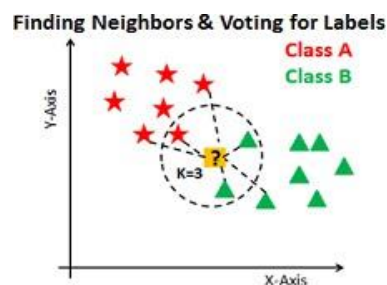


Fig 3.8 Plot of ideal KNN Algorithm

3.4.3 Advantages and Disadvantages

Advantages

- The algorithm is simple and easy to implement.
- There's no need to build a model, tune several parameters, or make additional assumptions.
- The algorithm is versatile. It can be used for classification, regression, and search
- The training phase of K-nearest neighbor classification is much faster compared to other classification algorithms. There is no need to train a model for generalization that is why KNN is known as the simple and instance-based learning algorithm.
- KNN can be useful in case of nonlinear data. It can be used with the regression problem. Output value for the object is computed by the average of k closest neighbors value.

Disadvantages

- The algorithm gets significantly slower as the number of examples and/or predictors/independent variables increase.
- The testing phase of K-nearest neighbor classification is slower and costlier in terms of time and memory. It requires large memory for storing the entire training dataset for prediction.
- KNN requires scaling of data because KNN uses the Euclidean distance between two data points to find nearest neighbors. Euclidean distance is sensitive to magnitudes. The features with high magnitudes will weigh more than features with low magnitudes.
- KNN also not suitable for large dimensional data.

```
def Diabet(self):
    import pandas as pd
    path="C:\\Users\\Admin\\Desktop\\ajay\\final report\\diabetes2.csv"
    data=pd.read_csv(path)
    print(data)
    import sklearn
    from sklearn.model_selection import train_test_split
    inputs=data.drop('Outcome','columns')
    output=data['Outcome']
    x_train,x_test,y_train,y_test=train_test_split(inputs,output,train_size=0.8)
    from sklearn.neighbors import KNeighborsClassifier
    model=KNeighborsClassifier(n_neighbors=27)
    model.fit(x_train,y_train)
    y_pred=model.predict(x_test)
    from sklearn.metrics import confusion_matrix
    cm=confusion_matrix(y_test,y_pred)
    print(cm)
    result = model.predict([[int(self.txtpreg.text()),int(self.txtglu.text()),int(self.txtbp.text()),
                             int(self.txtskinth.text()),int(self.txtinsu.text()),float(self.txtbmi.text()),
                             float(self.txtdpf.text()),int(self.txtage.text())]])
    self.lblmessage.setText(str(result))
```

diabetes

Diabetes

Pregnancies	1	Insulin	0
Glucose	126	BMI	30.1
BloodPressure	60	Age	47
SkinThickness	0		
DiabetesPedigreeFunction	0.349		

[0]

Fig 3.9 Example of KNN Algorithm

3.5 SVM

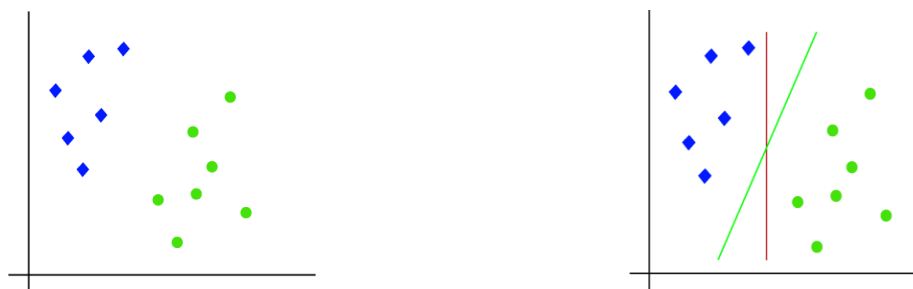
“Support Vector Machine” (SVM) is a supervised machine learning algorithm that can be used for both classification and regression challenges. However, it is mostly used in classification problems. In the SVM algorithm, we plot each data item as a point in n -dimensional space (where n is a number of features you have) with the value of each feature being the value of a particular coordinate.

3.5.1 Features of SVM

- SVM is a Supervised Learning algorithm that uses labelled input data set to predict the output of the data points.
- It is one of the simplest Machine learning algorithms and it can be easily implemented for a varied set of problems.
- SVM can be used for solving both classification and regression problems.

3.5.2 Working

The working of the SVM algorithm can be understood by using an example. Suppose we have a dataset that has two tags (green and blue), and the dataset has two features x_1 and x_2 . We want a classifier that can classify the pair(x_1 , x_2) of coordinates in either green or blue. So, as it is 2-d space so by just using a straight line, we can easily separate these two classes. But there can be multiple lines that can separate these classes. Consider the below image:



Hence, the SVM algorithm helps to find the best line or decision boundary; this best boundary or region is called as a **hyperplane**. SVM algorithm finds the closest point of the lines from both the classes. These points are called support vectors. The distance between the vectors and the hyperplane is called as **margin**. And the goal of SVM is to maximize this margin. The **hyperplane** with maximum margin is called the **optimal hyperplane**.

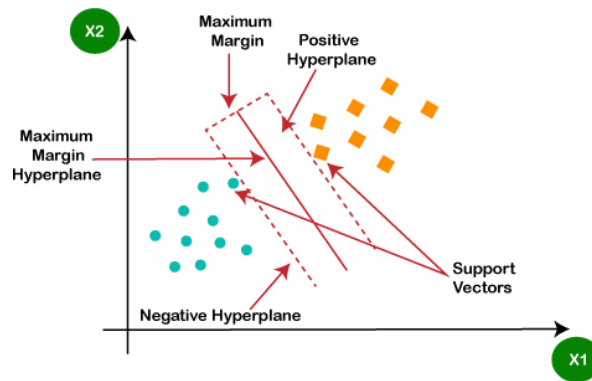


Fig 3.10 Plot of ideal SVM Algorithm

3.5.3 Advantages and Disadvantages

Advantages

- SVM works relatively well when there is a clear margin of separation between classes.
- SVM is more effective in high dimensional spaces.
- SVM is effective in cases where the number of dimensions is greater than the number of samples.
- SVM is relatively memory efficient.

Disadvantages

- SVM algorithm is not suitable for large data sets.
- SVM does not perform very well when the data set has more noise i.e. target classes are overlapping.
- In cases where the number of features for each data point exceeds the number of training data samples, the SVM will underperform.
- As the support vector classifier works by putting data points, above and below the classifying hyperplane there is no probabilistic explanation for the classification.

```
def winequality(self):
    import pandas as pd
    import sklearn
    path = "C:\\Users\\Admin\\Desktop\\ajay\\final report\\winequalitywhite.csv"
    data = pd.read_csv(path)
    print(data)
    data.fixedacidity = data.fixedacidity.fillna(data.fixedacidity.median())
    data.volatileacidity = data.volatileacidity.fillna(data.volatileacidity.median())
    data.citricacid = data.citricacid.fillna(data.citricacid.median())
    data.residualsugar = data.residualsugar.fillna(data.residualsugar.median())
    data.chlorides = data.chlorides.fillna(data.chlorides.median())
    data.freesulfurdioxide = data.freesulfurdioxide.fillna(data.freesulfurdioxide.median())
    data.totalsulfurdioxide = data.totalsulfurdioxide.fillna(data.totalsulfurdioxide.median())
    data.pH = data.pH.fillna(data.pH.median())
    data.density = data.density.fillna(data.density.median())
    data.sulphates = data.sulphates.fillna(data.sulphates.median())
    data.alcohol = data.alcohol.fillna(data.alcohol.median())
    print(data)
    inputs = data.drop(['type', 'quality'], 'columns')
    output = data['quality']
    import sklearn
    from sklearn.model_selection import train_test_split
    x_train, x_test, y_train, y_test = train_test_split(inputs, output, train_size=0.8)
    from sklearn.svm import SVC
    model = SVC()
    model.fit(x_train, y_train)
    y_pred = model.predict(x_test)
    from sklearn.metrics import confusion_matrix
    cm = confusion_matrix(y_test, y_pred)
    print(cm)
    result = model.predict(sc.fit_transform([[float(self.txtfa.text()), float(self.txtva.text()), float(self.txtca.text()),
                                             float(self.txtts.text()), float(self.txtc.text()), float(self.txtfs.text()),
                                             float(self.txttd.text()), float(self.txtph.text()),
                                             float(self.txts.text()), float(self.txta.text())]))

    self.lblmsg.setText(str(result))
```

wine

Wine Qualities

Fixed Acidity	7	Total Sulfurdioxide	186
Volatile Acidity	0.23	Density	0.99560
Citric Acid	0.32	pH	3.2
Residual Sugar	8.5	Sulphates	0.4
Chlorides	0.058	Alcohol	10
Free Sulfurdioxide	47		

Predict

[6]

Fig 3.11 Example of SVM Algorithm

3.6 Decision Tree

Decision Tree is a **supervised learning technique** that can be used for both classification and Regression problems, but mostly it is preferred for solving Classification problems. It is a tree-structured classifier, where **internal nodes represent the features of a dataset, branches represent the decision rules** and **each leaf node represents the outcome**. It is a graphical representation for getting all the possible solutions to a problem/decision on based on given conditions. In a Decision tree, there are two nodes, which are the **Decision Node** and **Leaf Node**. Decision nodes are used to make any decision and have multiple branches, whereas Leaf nodes are the output of those decisions and do not contain any further branches.

3.6.1 Features of Decision tree

- Decision Trees usually mimic human thinking ability while making a decision, so it is easy to understand.
- The logic behind the decision tree can be easily understood because it shows a tree-like structure.
- It is very easy to understand and implement.

3.6.2 Working

In a decision tree, for predicting the class of the given dataset, the algorithm starts from the root node of the tree. This algorithm compares the values of root attribute with the record (real dataset) attribute and, based on the comparison, follows the branch and jumps to the next node. For the next node, the algorithm again compares the attribute value with the other sub-nodes and move further. It continues the process until it reaches the leaf node of the tree. The complete process can be better understood using the below algorithm:

Step-1: Begin the tree with the root node, says S, which contains the complete dataset.

Step-2: Find the best attribute in the dataset using **Attribute Selection Measure (ASM)**.

Step-3: Divide the S into subsets that contains possible values for the best attributes.

Step-4: Generate the decision tree node, which contains the best attribute.

Step-5: Recursively make new decision trees using the subsets of the dataset created in step3.

Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.

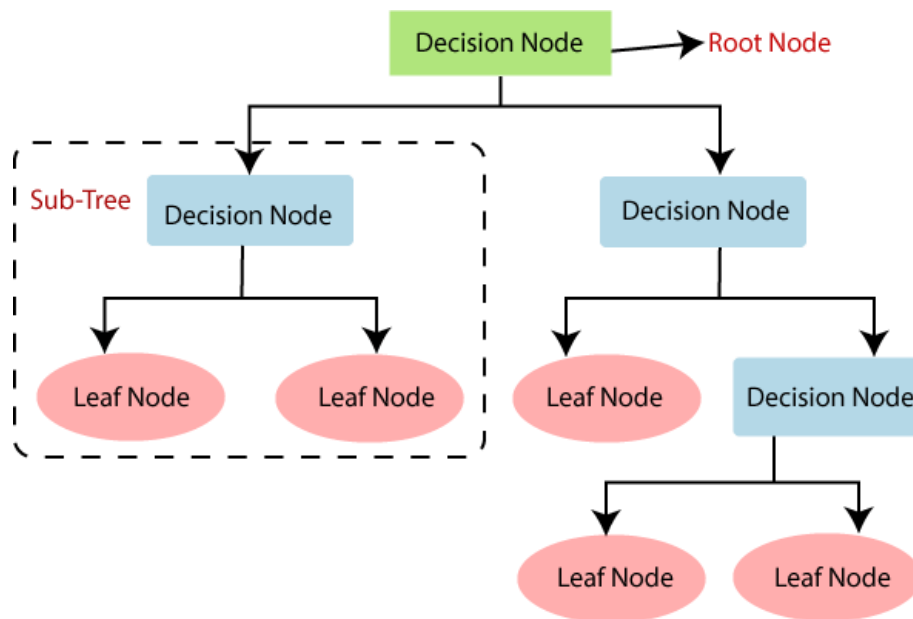


Fig 3.12 Ideal diagram of a Decision Tree

3.6.3 Advantages and Disadvantages

Advantages

- It is simple to understand as it follows the same process which a human follow while making any decision in real-life.
- It can be very useful for solving decision-related problems.
- It helps to think about all the possible outcomes for a problem.
- There is less requirement of data cleaning compared to other algorithms.

Disadvantages

- The decision tree contains lots of layers, which makes it complex.
- It may have an over fitting issue, which can be resolved using the **Random Forest algorithm**.
- For more class labels, the computational complexity of the decision tree may increase.

```
def loan(self):
    import pandas as pd
    path = "C:\\Users\\Admin\\Desktop\\ajay\\SVM\\loanpredictiondataset\\train.csv"
    data = pd.read_csv(path)
    print(data)
    inputs = data.drop('Loan_Status','columns')
    output = data['Loan_Status']

    import sklearn
    from sklearn.preprocessing import LabelEncoder

    le_Gender = LabelEncoder() # initialize my Label encoder
    inputs['Gender_n'] = le_Gender.fit_transform(inputs['Gender'])
    le_Married = LabelEncoder() # initialize my Label encoder
    inputs['Married_n'] = le_Married.fit_transform(inputs['Married'])
    le_Education = LabelEncoder() # initialize my Label encoder
    inputs['Education_n'] = le_Education.fit_transform(inputs['Education'])
    le_Self_Employed = LabelEncoder() # initialize my Label encoder
    inputs['Self_Employed_n'] = le_Self_Employed.fit_transform(inputs['Self_Employed'])
    le_Property_Area = LabelEncoder() # initialize my Label encoder
    inputs['Property_Area_n'] = le_Property_Area.fit_transform(inputs['Property_Area'])

    inputs = inputs.drop(['Gender', 'Married', 'Education', 'Self_Employed', 'Property_Area', 'Loan_ID'], 'columns') # removing columns
    print(inputs)
    print(output)

    from sklearn import tree
    model = tree.DecisionTreeClassifier() # init model
    model.fit(inputs, output) # train model
    Gender = self.txtgender.text()
    Married = self.txtmarried.text()
    Dependents = self.txtdependents.text()
    Education = self.txteducation.text()
    Self_Employed = self.txtselfemployed.text()
    ApplicantIncome = self.txtapplicantincome.text()
    CoapplicantIncome = self.txtcoapplicantincome.text()
    LoanAmount = self.txtloanamount.text()
    Loan_Amount_Term = self.txtloanamountterm.text()
    Credit_History = self.txtcredithistory.text()
    Property_Area = self.txtpropertyarea.text()

    ans = model.predict([[float(Gender), float(Married), float(Dependents), float(Education), float(Self_Employed),
                          float(ApplicantIncome), float(CoapplicantIncome), float(LoanAmount), float(Loan_Amount_Term),
                          float(Credit_History), float(Property_Area)]])

    if ans == 'Y':
        self.lblmessage.setText(str("YES"))
    elif ans == 'N':
        self.lblmessage.setText(str("NO"))
```

LOAN STATUS

GENDER	1	APPLICANT INCOME	5849
MARRIED	0	COAPPLICANT INCOME	0
DEPENDENTS	0	LOAN AMOUNT	266
EDUCATION	0	LOAN_AMOUNT_TERM	360
SELF_EMPLOYED	0	CREDIT_HISTORY	1
PROPERTY_AREA	2		

PREDICT

YES

LOAN STATUS

GENDER	1	APPLICANT INCOME	4583
MARRIED	1	COAPPLICANT INCOME	1508
DEPENDENTS	1	LOAN AMOUNT	128
EDUCATION	0	LOAN_AMOUNT_TERM	360
SELF_EMPLOYED	0	CREDIT_HISTORY	1
PROPERTY_AREA	0		

PREDICT

NO

Fig 3.13 Example of Decision Tree

CHAPTER 4

Project -1 Industrial-Residential Air quality classification

The Industrial-Residential Air Quality Classification using Machine Learning project focuses on analyzing city-based environmental data to classify air quality conditions in different areas such as industrial and residential zones. With increasing urbanization and pollution, monitoring air quality has become essential for public health and environmental safety. The dataset, which includes city-level parameters like pollutant concentrations (CO, NO₂, SO₂), temperature, humidity, and other environmental factors, is used to train Machine Learning models. These models learn patterns in the data and accurately classify air quality levels or zone types. By automating this process, the system provides quick and reliable predictions, helping authorities and individuals take necessary actions. This project demonstrates the practical application of AI in environmental monitoring and smart city development.

4.1 Problem Statement

Rapid urbanization and industrialization have led to increasing air pollution, making it difficult to monitor and differentiate air quality levels across industrial and residential areas. Traditional methods of analysis are slow and may not provide accurate classification of pollution levels. The problem is to develop a Machine Learning-based system that can classify air quality using city environmental data such as pollutant levels and weather conditions.

4.2 Dataset

The dataset used in the Industrial-Residential Air Quality Classification project contains city-based environmental data such as pollutant levels including carbon monoxide (CO), nitrogen dioxide (NO₂), sulfur dioxide (SO₂), and other air quality indicators. It may also include parameters like temperature, humidity, date, and location details to capture environmental variations. The dataset consists of numerical values representing pollutant concentrations and may include a target variable indicating the air quality category or zone type (industrial or residential). This data is typically collected from monitoring stations, and is preprocessed through cleaning, handling missing values, and normalization before being used to train Machine Learning models.

The overview of the original dataset is shown in fig 4.1., with its original features:

```
print(df.head())
```

	Date	City	CO	NO2	SO2	O3	PM2.5	PM10	\
0	2024-01-01 00:00:00+00:00	Moscow	208.0	15.9	13.2	44.0	8.6	9.4	
1	2024-01-01 01:00:00+00:00	Moscow	207.0	17.4	13.7	44.0	8.6	10.5	
2	2024-01-01 02:00:00+00:00	Moscow	217.0	19.0	15.5	43.0	10.4	12.9	
3	2024-01-01 03:00:00+00:00	Moscow	231.0	21.0	20.7	36.0	12.3	15.3	
4	2024-01-01 04:00:00+00:00	Moscow	263.0	34.5	27.2	27.0	13.6	20.0	

	Type
0	Industrial
1	Industrial
2	Industrial
3	Industrial
4	Industrial

Fig 4.1 Overview of Dataset

4.3 Algorithm –Multiple Linear Regression

The Industrial–Residential Air Quality Classification project uses Machine Learning algorithms to analyze environmental data and classify air quality levels. Common algorithms include Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and Logistic Regression. Among these, the Random Forest algorithm is widely used because it provides high accuracy and handles complex datasets effectively by combining multiple decision trees. The model is trained using features like pollutant levels (CO, NO₂, SO₂), temperature, and humidity, and then tested to evaluate its performance. The best-performing algorithm is selected based on accuracy and reliability for final predictions.

4.4 Programming Steps

- Import required libraries such as Pandas, NumPy, and Scikit-learn
- Load the air quality dataset (CSV file)
- Explore the dataset (check rows, columns, null values, data types)
- Perform data preprocessing (handle missing values, clean data)
- Encode categorical variables (if any) using Label Encoding
- Split the dataset into features (X) and target variable (y)
- Divide the data into training and testing sets (e.g., 80:20 ratio)

- Select a Machine Learning algorithm (e.g., Random Forest)
- Train the model using training data
- Test the model using test data and calculate accuracy
- Save the trained model using pickle
- Use the model to predict air quality classification for new data

Code:

Air Quality Classification

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import pickle

# Load dataset
data = pd.read_csv("E:\Internship_project\Dataset\Air-quality-Classification\City_Types.csv")
# change file name if needed

# Preprocessing
data = data.dropna()
le = LabelEncoder()
for col in data.select_dtypes(include='object').columns:
    data[col] = le.fit_transform(data[col])

# Features & Target
X = data.drop("target", axis=1) # replace 'target' with your column
y = data["target"]

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Model Training
model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)

# Prediction & Accuracy
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))

# Save Model
pickle.dump(model, open("air_model.pkl", "wb"))

# Sample Prediction
print("Prediction:", model.predict([X.iloc[0]]))
```

Output:

```
# Print all accuracies
print("\n=== Model Accuracies ===")
for name, acc in accuracies.items():
    print(f"{name}: {acc:.4f}")

# Find best algorithm
best_model = max(accuracies, key=accuracies.get)
best_accuracy = accuracies[best_model]

print("\n=== Best Model ===")
print(f"Best Algorithm: {best_model}")
print(f"Accuracy: {best_accuracy:.4f}")

=== Model Accuracies ===
KNN: 1.0000
Random Forest: 1.0000
Decision Tree: 1.0000
Logistic Regression: 1.0000

=== Best Model ===
Best Algorithm: KNN
Accuracy: 1.0000
```

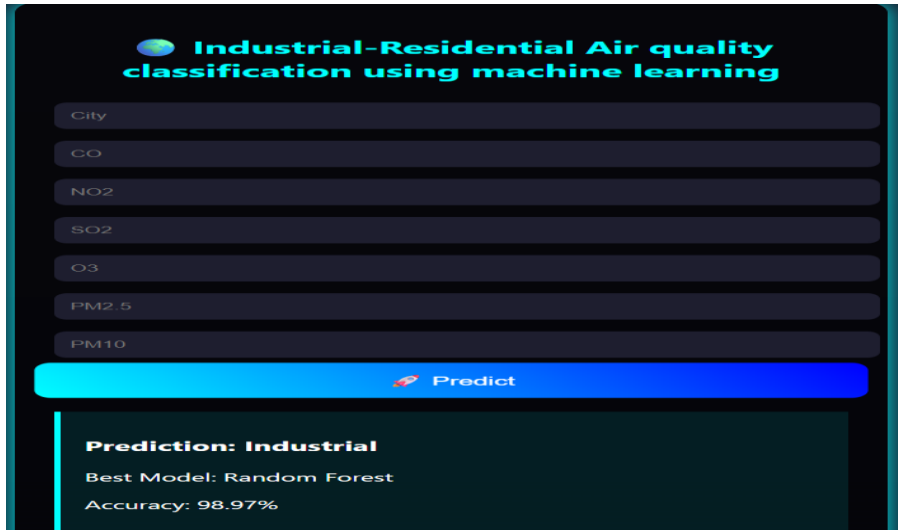
Fig 4.2 Accuracy Prediction and Model Comparision

Implementing Code using Django:

```
return render(request, "city.html", {
    "result": result,
    "best_model": best_model_name,
    "accuracy": round(best_accuracy * 100, 2),
    "model_results": model_results,
    "input_data": input_data_display,
    "error": error
})
```

Fig 4.3 Connecting views.py and city.html file

Output:



Industrial-Residential Air quality classification using machine learning

City

CO

NO2

SO2

O3

PM2.5

PM10

Predict

Prediction: Industrial

Best Model: Random Forest

Accuracy: 98.97%

Fig 4.4 Dashboard and City pollution Prediction

4.5 CONCLUSION

The Industrial–Residential Air Quality Classification using Machine Learning project provides an effective solution for analyzing and classifying air quality based on environmental data. By using algorithms like Random Forest, the system can accurately distinguish between different air quality conditions in industrial and residential areas. This helps in better monitoring of pollution levels and supports informed decision-making for environmental management. Overall, the project demonstrates how Machine Learning can be applied to real-world problems, improving public health awareness and contributing to smarter and cleaner urban development.

CHAPTER 5

Project -2

Plant communication classification

The **Plant Communication Classification using Machine Learning** project focuses on analyzing plant-related data to identify and classify different plant conditions or communication patterns. Plants respond to environmental changes such as air quality, temperature, humidity, and pollutants through measurable signals, which can be captured using sensor data. However, manually analyzing these patterns is complex and inefficient. In this project, Machine Learning techniques are used to process the dataset, identify patterns, and classify plant responses accurately. The model is trained using features extracted from the dataset (such as environmental parameters or signal values) and predicts the category or condition of the plant. By using algorithms like Random Forest, Decision Tree, or other classifiers, the system improves accuracy and automates the classification process. This project demonstrates how AI can be applied to environmental monitoring and smart agriculture, helping in early detection of plant stress and improving overall plant health management.

5.1 Problem Statement

Plants respond to environmental changes such as air quality, temperature, and humidity through measurable signals, but identifying and classifying these responses manually is difficult and time-consuming. Traditional methods lack accuracy and efficiency in analyzing complex plant-related data. The problem is to develop a Machine Learning model that can automatically classify plant conditions or communication patterns based on sensor and environmental data.

5.2 Dataset

The dataset used in the Plant Communication Classification project consists of environmental and plant-related parameters such as air quality levels, temperature, humidity, and other sensor readings that reflect plant responses. It may include features like gas concentrations (CO, NO₂, SO₂), time-based measurements, and signal values collected from monitoring systems. The dataset contains both numerical and categorical data, with the target variable representing the plant condition or classification category. It is preprocessed through cleaning, handling missing values, and normalization before being used to train the Machine Learning model.

5.3 Algorithms

- Uses Machine Learning algorithms like Random Forest, Decision Tree, KNN, and SVM
- Random Forest is preferred for higher accuracy and stability
- Works on both numerical and categorical data effectively
- Handles complex and large datasets efficiently
- Uses multiple decision trees (ensemble method) for better prediction
- Reduces overfitting compared to single models
- Provides reliable and consistent classification results
- Trained using labeled dataset (supervised learning)
- Evaluated using accuracy and performance metrics
- Suitable for real-time plant condition classification and prediction

5.4 Programming Steps

- Import required libraries like NumPy, Pandas, and Scikit-learn
- Load the dataset (CSV file) into a DataFrame
- Explore and understand the dataset (check shape, columns, null values)
- Perform data preprocessing (handle missing values, encode categorical data)
- Split the dataset into features (X) and target variable (y)
- Divide data into training and testing sets (e.g., 80:20 split)
- Select and apply Machine Learning algorithm (e.g., Random Forest)
- Train the model using training data
- Test the model using test data and evaluate accuracy
- Save the trained model using pickle
- Use the model to make predictions on new input data
- (Optional) Integrate the model into a web app using Django

Code:

```
# Plant Communication Classification using ML

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import pickle

# Load dataset
data = pd.read_csv("E:\Internship_project\ModelTraining\Plant_classification.ipynb")

# Preprocessing
data = data.dropna()
le = LabelEncoder()
for col in data.select_dtypes(include='object').columns:
    data[col] = le.fit_transform(data[col])

# Features & Target
X = data.drop("target", axis=1) # change 'target' if needed
y = data["target"]

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Model Training
model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)

# Prediction & Accuracy
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))

# Save Model
pickle.dump(model, open("plant_model.pkl", "wb"))

# Sample Prediction
print("Prediction:", model.predict([X.iloc[0]]))
```

Output:

```
# -----  
# Best Model  
# -----  
  
best_model_name = max(accuracies, key=accuracies.get)  
best_model = models[best_model_name]  
  
print("\n 🏆 Best Model:")  
print(f"{best_model_name} with accuracy {accuracies[best_model_name]:.4f}")  
  
# -----  
# Accuracy Comparison Plot  
# -----  
  
colors = ['green' if name == best_model_name else 'skyblue' for name in accuracies]  
  
plt.figure()  
plt.bar(accuracies.keys(), accuracies.values(), color=colors)  
plt.title("Model Comparison (Best Highlighted)")  
plt.xticks(rotation=30)  
plt.ylabel("Accuracy")  
plt.show()
```

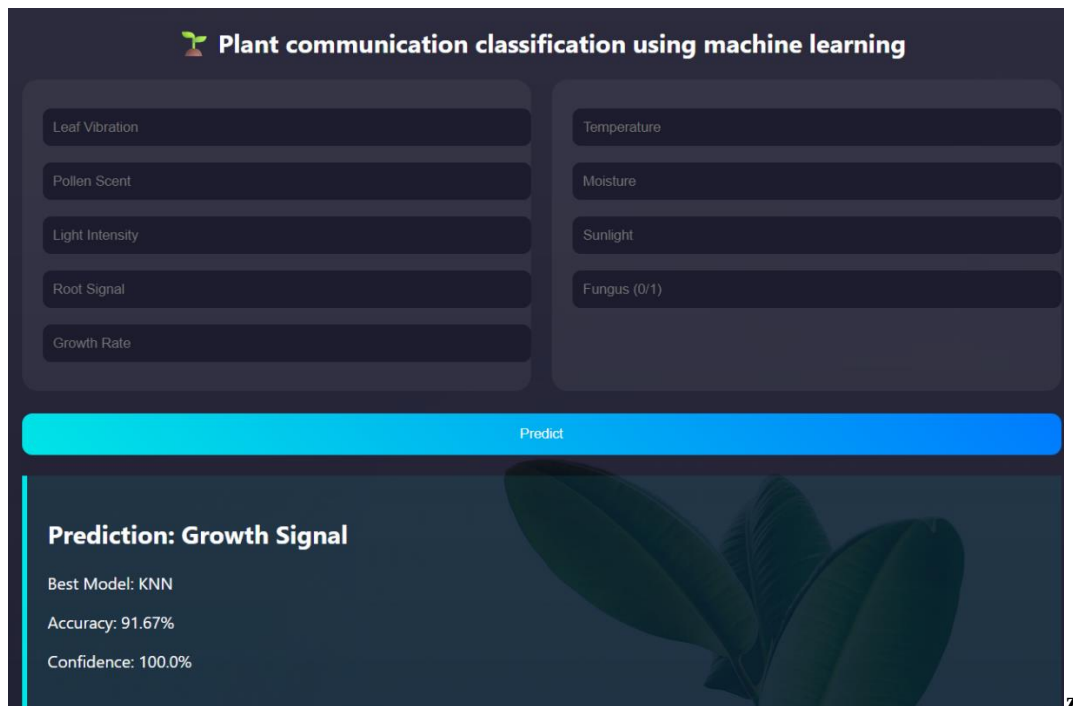
Fig 5.2 Accuracy Comparision

Implementing Code in Django:

```
if request.method == "POST":  
    try:  
        # ===== INPUT =====  
        lv = get_float(request.POST.get("vibration"))  
        pollen = get_float(request.POST.get("scent"))  
        bio = get_float(request.POST.get("light"))  
        root = get_float(request.POST.get("root"))  
        growth = get_float(request.POST.get("growth"))  
        temp = get_float(request.POST.get("temp"))  
        moisture = get_float(request.POST.get("moisture"))  
        sunlight = get_float(request.POST.get("sunlight"))  
        fungus = int(request.POST.get("fungus") or 0)  
  
        # ===== LOAD DATASET =====  
        df = pd.read_csv(r"E:\Internship_project\Dataset\Plant-communication-classification\plant_data.csv")  
        df = df.dropna()  
  
        df.columns = [  
            "vibration", "scent", "light", "root",  
            "growth", "temp", "moisture",  
            "sunlight", "fungus", "target"  
        ]
```

Fig 5.3 Implementing Using Django

Output:



The screenshot displays a web application titled "Plant communication classification using machine learning". It features a dark-themed interface with two columns of input fields. The left column includes "Leaf Vibration", "Pollen Scent", "Light Intensity", "Root Signal", and "Growth Rate". The right column includes "Temperature", "Moisture", "Sunlight", and "Fungus (0/1)". A prominent blue "Predict" button is centered below the input fields. The output section, highlighted with a blue border, shows the "Prediction: Growth Signal" and provides model details: "Best Model: KNN", "Accuracy: 91.67%", and "Confidence: 100.0%". A faint image of a plant is visible in the background of the output area.

Fig 5.4 Plant communication Prediction System

5.5 CONCLUSION

The Plant Communication Classification using Machine Learning project successfully demonstrates how AI can be used to analyze and classify plant conditions based on environmental and sensor data. By applying algorithms like Random Forest, the system can accurately identify patterns and predict plant responses. This helps in early detection of plant stress and improves monitoring efficiency. Overall, the project provides a smart, automated, and reliable solution for understanding plant behavior, contributing to better agricultural practices and environmental management.

CHAPTER 6

Project 3 Smart Fertilizer Recommendation

The **Smart Fertilizer Recommendation System** uses Machine Learning algorithms to analyze agricultural data such as soil nutrients, crop type, and environmental conditions to make intelligent decisions. Instead of following fixed rules, the system learns patterns from the dataset and predicts the most suitable fertilizer for given inputs. Algorithms like Random Forest, Decision Tree, and K-Nearest Neighbors are part of AI techniques that enable the system to improve accuracy and provide data-driven recommendations. Since the model is trained on real data and makes predictions automatically, it falls under the domain of **Artificial Intelligence and Data Science**. Additionally, integrating the trained model into a Django web application makes it a complete AI-based real-world solution.

6.1 Problem Statement

Farmers often rely on traditional knowledge or guesswork to select fertilizers, which can lead to incorrect usage and reduced crop yield. This results in soil degradation, increased costs, and inefficient farming practices. The problem is to develop an AI-based system that recommends the most suitable fertilizer based on soil nutrients, crop type, and environmental conditions.

6.2 Dataset

The dataset for the Smart Fertilizer Recommendation System includes input features like temperature, humidity, moisture, soil type, crop type, and nutrient levels (N, P, K), along with the target output which is the recommended fertilizer. It contains both numerical and categorical data. The data is preprocessed through cleaning, encoding, and normalization before being used to train the Machine Learning model.

```
Click to add a breakpoint =====
# 12. SAVE MODEL + OBJECTS
# =====
model.save("fertilizer_model.h5")

pickle.dump(scaler, open("scaler.pkl", "wb"))
pickle.dump(encoders, open("encoders.pkl", "wb"))
pickle.dump(target_encoder, open("target_encoder.pkl", "wb"))

print("✅ Model + Encoders Saved Successfully!")

e:\Internship_project\.venv\lib\site-packages\keras\src\layers\core\dense.py:95: UserWarning: Do not pass an `input_shape`
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/50
500/500 ————— 4s 3ms/step - accuracy: 0.6611 - loss: 0.9627 - val_accuracy: 0.7890 - val_loss: 0.6463
Epoch 2/50
500/500 ————— 2s 4ms/step - accuracy: 0.7739 - loss: 0.6640 - val_accuracy: 0.7995 - val_loss: 0.5833
Epoch 3/50
500/500 ————— 2s 3ms/step - accuracy: 0.7983 - loss: 0.5917 - val_accuracy: 0.8110 - val_loss: 0.5358
Epoch 4/50
500/500 ————— 1s 3ms/step - accuracy: 0.8091 - loss: 0.5562 - val_accuracy: 0.8215 - val_loss: 0.5045
Epoch 5/50
500/500 ————— 1s 3ms/step - accuracy: 0.8151 - loss: 0.5204 - val_accuracy: 0.8260 - val_loss: 0.4855
```

Fig 6.1 Dataset Training Using CNN model

Solution:

The solution for the Smart Fertilizer Recommendation System involves collecting and preprocessing agricultural data by cleaning, encoding categorical values, and normalizing features to ensure better performance. Machine Learning algorithms such as Decision Tree, Random Forest, and K-Nearest Neighbors are applied to train the model using inputs like soil nutrients, crop type, and environmental conditions. The best-performing model is selected based on accuracy and used to build a prediction system that recommends the most suitable fertilizer. This model is then integrated into a Django web application, where users can enter their data through a simple interface and receive instant recommendations. Overall, the system provides an efficient, accurate, and user-friendly solution that supports cost-effective and sustainable farming practices.

CODE:

```
import pandas as pd
import numpy as np
import pickle

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.utils import to_categorical

# =====
# 1. LOAD DATASET
# =====
df = pd.read_csv(r"E:\Internship_project\my_ai_project\my_ai_project\crop\crop-training\fertilizer_recommendation.csv")

# =====
# 2. HANDLE CATEGORICAL DATA
# =====
categorical_cols = [
    "Soil_Type", "Crop_Type", "Crop_Growth_Stage",
    "Season", "Irrigation_Type", "Previous_Crop",
    "Region", "Fertilizer_Used_Last_Season"
]

encoders = {}

for col in categorical_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    encoders[col] = le
```

```
Click to add a breakpoint =====
# 12. SAVE MODEL + OBJECTS
# =====
model.save("fertilizer_model.h5")

pickle.dump(scaler, open("scaler.pkl", "wb"))
pickle.dump(encoders, open("encoders.pkl", "wb"))
pickle.dump(target_encoder, open("target_encoder.pkl", "wb"))

print("✅ Model + Encoders Saved Successfully!")
```

```
e:\Internship_project\.venv\lib\site-packages\keras\src\layers\core\dense.py:95: UserWarning: Do not pass an `input_shape`
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/50
500/500 ————— 4s 3ms/step - accuracy: 0.6611 - loss: 0.9627 - val_accuracy: 0.7890 - val_loss: 0.6463
Epoch 2/50
500/500 ————— 2s 4ms/step - accuracy: 0.7739 - loss: 0.6640 - val_accuracy: 0.7995 - val_loss: 0.5833
Epoch 3/50
500/500 ————— 2s 3ms/step - accuracy: 0.7983 - loss: 0.5917 - val_accuracy: 0.8110 - val_loss: 0.5358
Epoch 4/50
500/500 ————— 1s 3ms/step - accuracy: 0.8091 - loss: 0.5562 - val_accuracy: 0.8215 - val_loss: 0.5045
Epoch 5/50
500/500 ————— 1s 3ms/step - accuracy: 0.8151 - loss: 0.5204 - val_accuracy: 0.8260 - val_loss: 0.4855
```

Output:



Smart Fertilizer Recommendation using AI

Soil Information

Soil Type

Soil pH

Soil Moisture

Nitrogen Level

Phosphorus Level

Potassium Level

Environment & Crop

Temperature

Humidity

Crop Type

Predict Fertilizer

Recommended Fertilizer: NPK

Confidence: 41.85%

Fig 6.2 Output of Smart Fertilizer Recommendation

6.3 CONCLUSION

The Smart Fertilizer Recommendation System provides an effective AI-based solution for selecting the most suitable fertilizer using soil and environmental data. It helps farmers make accurate, data-driven decisions, improving crop yield and reducing unnecessary fertilizer usage. By integrating Machine Learning with a user-friendly application, the system simplifies the decision-making process and supports sustainable farming practices. Overall, this project demonstrates how AI can be used to solve real-world agricultural problems efficiently.

REFERENCES

- [1] www.karunadutechnologies.com
- [2] <https://www.simplilearn.com/10-algorithms-machine-learning-engineers-need-to-know-article>
- [3] <https://www.edureka.co/blog/machine-learning-algorithms/>
- [4] <https://www.analyticsvidhya.com/blog/2017/09/common-machine-learning-algorithms/>